Machine Learning Regression Analysis: Complete Experiment

Dataset: Salary vs Years of Experience

This notebook performs:

- Simple Linear Regression
- Polynomial Regression (degrees 2-7)
- Random Forest Regression
- Comprehensive comparison and evaluation

Import Required Libraries

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import warnings
warnings.filterwarnings('ignore')

# Set style for better visualizations
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette('husl')

print("All libraries imported successfully!")
```

```
All libraries imported successfully!
```

Load Dataset from Local File

Loading data from data.dat file containing x_list and y_list.

In [2]:
```
# Load dataset from local data.dat file
import os

# Execute the data.dat file to get x_list and y_list
with open('data.dat', 'r') as f:
    exec(f.read())

# Create DataFrame from the lists
df = pd.DataFrame({
    'x': x_list,
    'y': y_list
})
```

```
print("Dataset loaded successfully from data.dat!")
print(f"\nDataset shape: {df.shape}")
print(f"X range: {df['x'].min()} to {df['x'].max()}")
print(f"Y range: {df['y'].min():.2e} to {df['y'].max():.2e}")
```

```
Dataset loaded successfully from data.dat!

Dataset shape: (605, 2)
X range: 175 to 8590
Y range: 2.54e+10 to 1.72e+12
```

Data Exploration and Cleaning

In [3]:
```
# Display first few rows
print("First 5 rows of the dataset:")
print(df.head())

print("\nDataset Information:")
print(df.info())

print("\nBasic Statistics:")
print(df.describe())

# Check for missing values
print("\nMissing Values:")
print(df.isnull().sum())
```

```
First 5 rows of the dataset:
     x            y
0  175  27012159310
1  180  30879471051
2  185  29343547254
3  190  34680123880
4  195  36018006609

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 605 entries, 0 to 604
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   x       605 non-null    int64
 1   y       605 non-null    int64
dtypes: int64(2)
memory usage: 9.6 KB
None

Basic Statistics:
                 x             y
count   605.000000  6.050000e+02
mean   3908.570248  4.352410e+11
std    2375.385988  5.051046e+11
min     175.000000  2.538102e+10
25%    1975.000000  4.451664e+10
50%    3225.000000  1.169459e+11
75%    5675.000000  1.002694e+12
max    8590.000000  1.723825e+12

Missing Values:
x    0
y    0
dtype: int64
```

In [4]:

```python
# Handle missing values if any
if df.isnull().sum().sum() > 0:
    print("Handling missing values...")
    df = df.dropna()  # Drop rows with missing values
    print(f"Dataset shape after cleaning: {df.shape}")
else:
    print("No missing values found. Dataset is clean!")

# Prepare X and y
X = df.iloc[:, 0].values.reshape(-1, 1)  # Independent variable
y = df.iloc[:, 1].values  # Dependent variable

print(f"\nX shape: {X.shape}")
print(f"y shape: {y.shape}")
```

```
No missing values found. Dataset is clean!

X shape: (605, 1)
y shape: (605,)
```

PART A-C: Simple Linear Regression

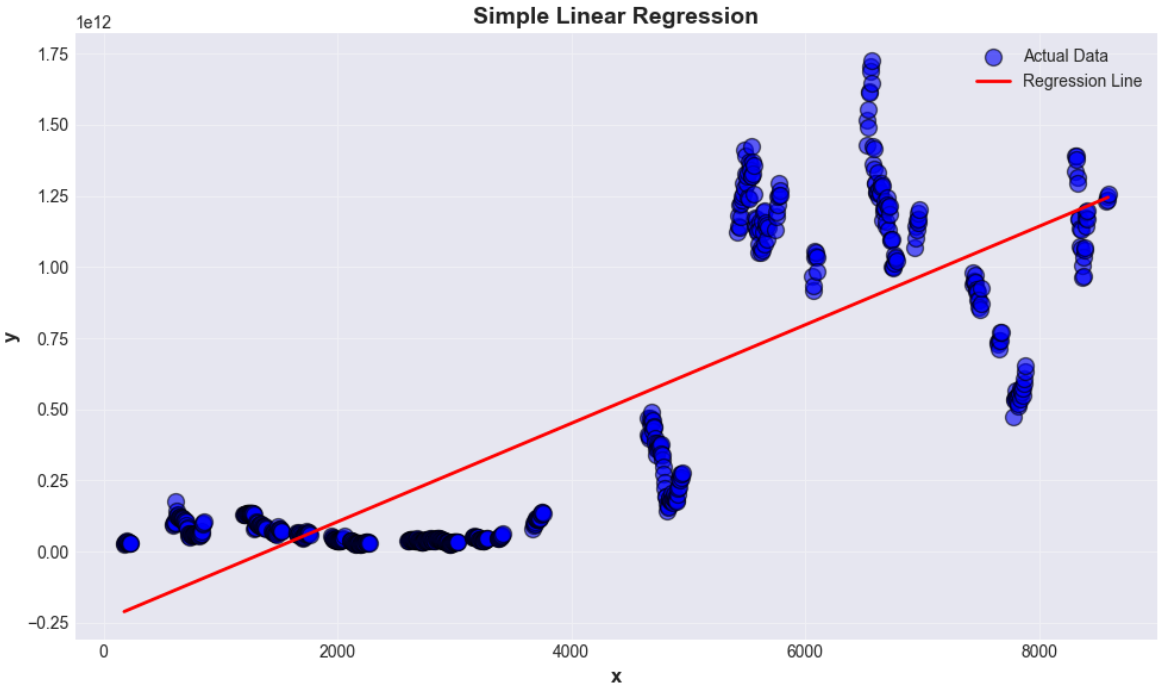
A) Train Simple Linear Regression Model

In [5]:
```
# Train Linear Regression Model
linear_model = LinearRegression()
linear_model.fit(X, y)

# Get coefficients
beta_0 = linear_model.intercept_
beta_1 = linear_model.coef_[0]

# Print the linear equation
print("="*60)
print("SIMPLE LINEAR REGRESSION EQUATION")
print("="*60)
print(f"\ny = {beta_0:.2f} + {beta_1:.2f}x\n")
print(f"Intercept (β₀): {beta_0:.2f}")
print(f"Slope (β₁): {beta_1:.2f}")
print("="*60)
```

```
============================================================
SIMPLE LINEAR REGRESSION EQUATION
============================================================

y = -240878642464.73 + 172983866.82x

Intercept (β₀): -240878642464.73
Slope (β₁): 172983866.82
============================================================
```

B) Visualization: Scatter Plot + Regression Line

In [6]:
```
# Predict values
y_pred_linear = linear_model.predict(X)

# Create visualization
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='blue', alpha=0.6, s=100, edgecolors='black', label='Act
plt.plot(X, y_pred_linear, color='red', linewidth=2, label='Regression Line')
plt.xlabel(df.columns[0], fontsize=12, fontweight='bold')
plt.ylabel(df.columns[1], fontsize=12, fontweight='bold')
plt.title('Simple Linear Regression', fontsize=14, fontweight='bold')
plt.legend(fontsize=10)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



C) Evaluate Linear Regression Model

In [7]:
```
# Calculate metrics
mae_linear = mean_absolute_error(y, y_pred_linear)
mse_linear = mean_squared_error(y, y_pred_linear)
rmse_linear = np.sqrt(mse_linear)
r2_linear = r2_score(y, y_pred_linear)

# Print results
print("="*60)
print("SIMPLE LINEAR REGRESSION - EVALUATION METRICS")
print("="*60)
print(f"Mean Absolute Error (MAE):       {mae_linear:.2f}")
print(f"Mean Squared Error (MSE):        {mse_linear:.2f}")
print(f"Root Mean Squared Error (RMSE):  {rmse_linear:.2f}")
print(f"R² Score:                        {r2_linear:.4f}")
print("="*60)
```

```
============================================================
SIMPLE LINEAR REGRESSION - EVALUATION METRICS
============================================================
Mean Absolute Error (MAE):       244863824375.89
Mean Squared Error (MSE):        86146415616265184870400.00
Root Mean Squared Error (RMSE):  293507096364.41
R² Score:                        0.6618
============================================================
```

PART D-F: Polynomial Regression (Degrees 2-7)

D, E, F) Train, Visualize, and Evaluate Polynomial Models

In [8]:
```
# Dictionary to store results
poly_results = []
poly_models = {}

degrees = [2, 3, 4, 5, 6, 7]

for degree in degrees:
    print(f"\n{'='*60}")
    print(f"POLYNOMIAL REGRESSION - DEGREE {degree}")
    print(f"{'='*60}")

    # D) Train Polynomial Regression
    poly_features = PolynomialFeatures(degree=degree)
    X_poly = poly_features.fit_transform(X)

    poly_model = LinearRegression()
    poly_model.fit(X_poly, y)

    # Store model
    poly_models[degree] = (poly_features, poly_model)

    # Print polynomial equation
    coefficients = poly_model.coef_
    intercept = poly_model.intercept_

    equation = f"y = {intercept:.2f}"
    for i in range(1, len(coefficients)):
        if coefficients[i] >= 0:
            equation += f" + {coefficients[i]:.2f}x^{i}"
        else:
            equation += f" - {abs(coefficients[i]):.2f}x^{i}"

    print(f"\nPolynomial Equation (Degree {degree}):")
    print(equation)

    # Predict values
    y_pred_poly = poly_model.predict(X_poly)

    # F) Evaluate model
    mae_poly = mean_absolute_error(y, y_pred_poly)
    mse_poly = mean_squared_error(y, y_pred_poly)
    rmse_poly = np.sqrt(mse_poly)
    r2_poly = r2_score(y, y_pred_poly)

    print(f"\nEvaluation Metrics:")
    print(f"  MAE:   {mae_poly:.2f}")
    print(f"  MSE:   {mse_poly:.2f}")
    print(f"  RMSE:  {rmse_poly:.2f}")
    print(f"  R²:    {r2_poly:.4f}")

    # Store results
    poly_results.append({
        'Model': f'Polynomial (Degree {degree})',
        'MAE': mae_poly,
```

```
        'MSE': mse_poly,
        'RMSE': rmse_poly,
        'R2_Score': r2_poly
    })

print("\n" + "="*60)
print("All polynomial models trained successfully!")
print("="*60)
```

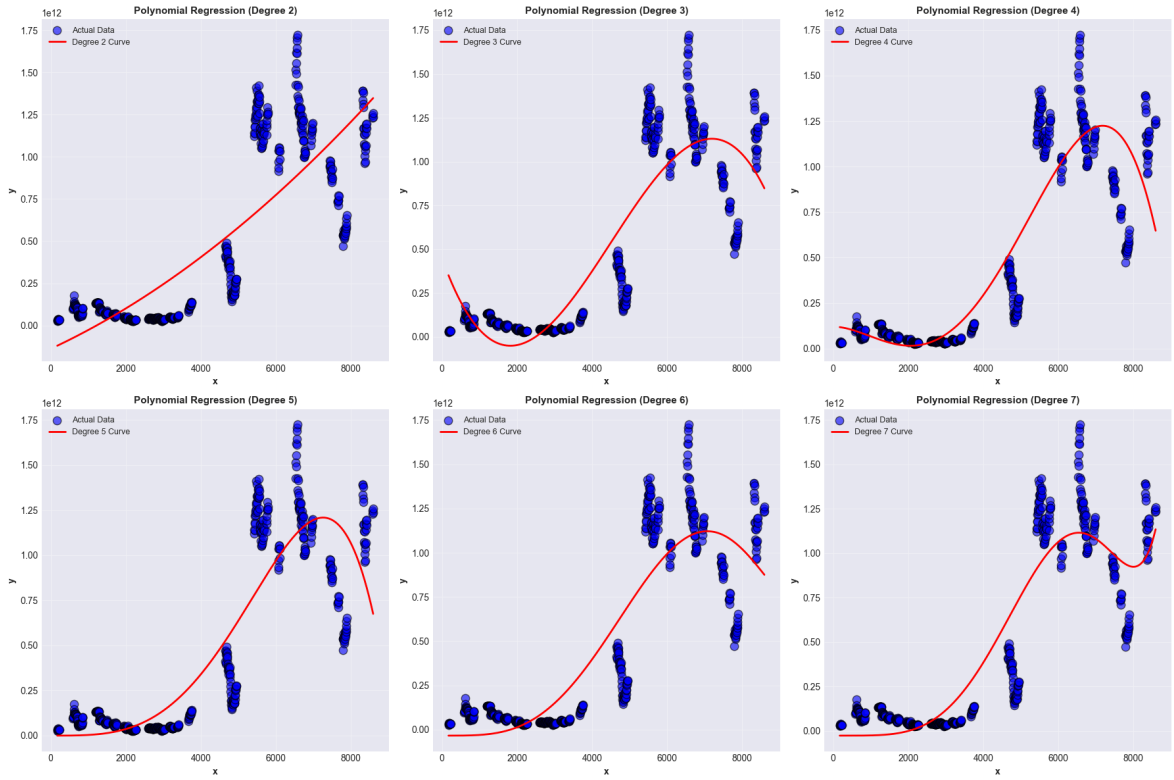
```
============================================================
POLYNOMIAL REGRESSION - DEGREE 2
============================================================

Polynomial Equation (Degree 2):
y = -138859191351.80 + 103478565.09x^1 + 8113.17x^2

Evaluation Metrics:
  MAE:   232153221668.74
  MSE:   84526296413178244890624.00
  RMSE:  290734064762.25
  R²:    0.6681

============================================================
POLYNOMIAL REGRESSION - DEGREE 3
============================================================

Polynomial Equation (Degree 3):
y = 449988153792.14 - 600980874.51x^1 + 206658.11x^2 - 15.29x^3

Evaluation Metrics:
  MAE:   173852856638.03
  MSE:   54601327213498033242112.00
  RMSE:  233669268868.41
  R²:    0.7856

============================================================
POLYNOMIAL REGRESSION - DEGREE 4
============================================================

Polynomial Equation (Degree 4):
y = 120420410172.04 - 38.93x^1 - 86551.31x^2 + 36.06x^3 - 0.00x^4

Evaluation Metrics:
  MAE:   151033100153.85
  MSE:   52216956628957139042304.00
  RMSE:  228510298737.18
  R²:    0.7950

============================================================
POLYNOMIAL REGRESSION - DEGREE 5
============================================================

Polynomial Equation (Degree 5):
y = -89756572.99 + 0.00x^1 + 0.00x^2 + 2.17x^3 + 0.00x^4 - 0.00x^5

Evaluation Metrics:
  MAE:   174786100090.15
  MSE:   57205035639954449891328.00
  RMSE:  239175742164.53
  R²:    0.7754

============================================================
POLYNOMIAL REGRESSION - DEGREE 6
============================================================

Polynomial Equation (Degree 6):
y = -34089220218.40 + 0.00x^1 + 0.00x^2 + 0.00x^3 + 0.00x^4 - 0.00x^5 + 0.00x^6

Evaluation Metrics:
```

```
  MAE:   179197866202.45
  MSE:   53805797384919405559808.00
  RMSE:  231960766908.80
  R²:    0.7888

============================================================
POLYNOMIAL REGRESSION - DEGREE 7
============================================================

Polynomial Equation (Degree 7):
y = -26827361396.71 - 0.00x^1 + 0.00x^2 + 0.00x^3 + 0.00x^4 + 0.00x^5 - 0.00x^6 +
0.00x^7

Evaluation Metrics:
  MAE:   156668082843.58
  MSE:   43455424986918080741376.00
  RMSE:  208459648342.11
  R²:    0.8294

============================================================
All polynomial models trained successfully!
============================================================
```

E) Visualizations for All Polynomial Degrees

In [9]:

```python
# Create smooth curve for visualization
X_smooth = np.linspace(X.min(), X.max(), 300).reshape(-1, 1)

# Create subplots
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
axes = axes.ravel()

for idx, degree in enumerate(degrees):
    poly_features, poly_model = poly_models[degree]

    # Predict smooth curve
    X_smooth_poly = poly_features.transform(X_smooth)
    y_smooth_pred = poly_model.predict(X_smooth_poly)

    # Plot
    axes[idx].scatter(X, y, color='blue', alpha=0.6, s=80, edgecolors='black', l
    axes[idx].plot(X_smooth, y_smooth_pred, color='red', linewidth=2, label=f'De
    axes[idx].set_xlabel(df.columns[0], fontsize=10, fontweight='bold')
    axes[idx].set_ylabel(df.columns[1], fontsize=10, fontweight='bold')
    axes[idx].set_title(f'Polynomial Regression (Degree {degree})', fontsize=11,
    axes[idx].legend(fontsize=9)
    axes[idx].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



Polynomial Regression Results Comparison Table

In [10]:
```
# Create comparison DataFrame
poly_comparison_df = pd.DataFrame(poly_results)
print("\n" + "="*80)
print("POLYNOMIAL REGRESSION - COMPARISON TABLE")
print("="*80)
print(poly_comparison_df.to_string(index=False))
print("="*80)
```

```
================================================================================
POLYNOMIAL REGRESSION - COMPARISON TABLE
================================================================================
                Model          MAE          MSE         RMSE  R2_Score
Polynomial (Degree 2) 2.321532e+11 8.452630e+22 2.907341e+11  0.668146
Polynomial (Degree 3) 1.738529e+11 5.460133e+22 2.336693e+11  0.785633
Polynomial (Degree 4) 1.510331e+11 5.221696e+22 2.285103e+11  0.794994
Polynomial (Degree 5) 1.747861e+11 5.720504e+22 2.391757e+11  0.775410
Polynomial (Degree 6) 1.791979e+11 5.380580e+22 2.319608e+11  0.788756
Polynomial (Degree 7) 1.566681e+11 4.345542e+22 2.084596e+11  0.829392
================================================================================
```

PART G-I: Random Forest Regression

G) Train Random Forest Regressor

In [11]:
```
# Train Random Forest Regressor
rf_model = RandomForestRegressor(n_estimators=100, max_depth=10, random_state=42
```

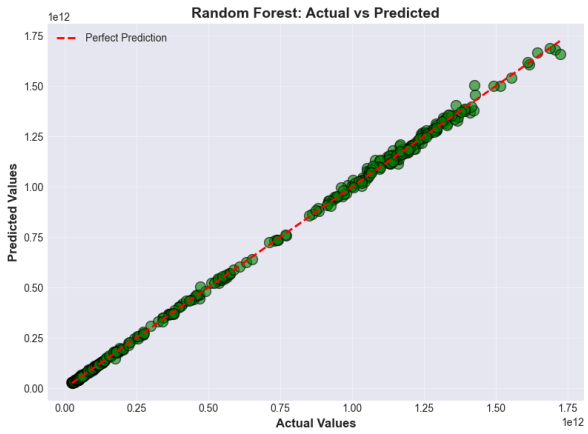
```
rf_model.fit(X, y)

# Print model details
print("="*60)
print("RANDOM FOREST REGRESSOR - MODEL DETAILS")
print("="*60)
print(f"Number of Trees (n_estimators):  {rf_model.n_estimators}")
print(f"Max Depth:                        {rf_model.max_depth}")
print(f"Random State:                     {rf_model.random_state}")
print(f"Number of Features:               {rf_model.n_features_in_}")
print("="*60)
```

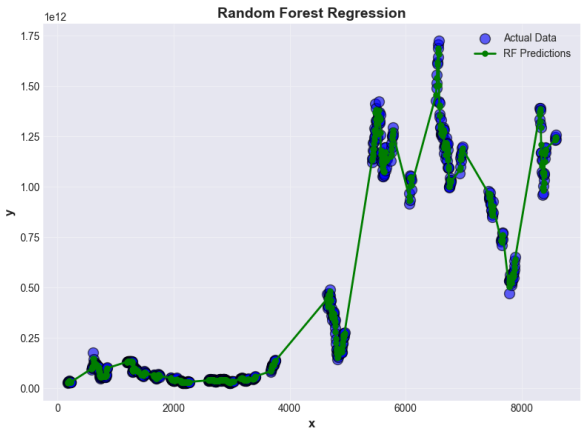
```
============================================================
RANDOM FOREST REGRESSOR - MODEL DETAILS
============================================================
Number of Trees (n_estimators):  100
Max Depth:                        10
Random State:                     42
Number of Features:               1
============================================================
```

H) Visualization: Actual vs Predicted Values

In [12]:

```
# Predict values
y_pred_rf = rf_model.predict(X)

# Create visualization
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Plot 1: Actual vs Predicted scatter
axes[0].scatter(y, y_pred_rf, color='green', alpha=0.6, s=100, edgecolors='black
axes[0].plot([y.min(), y.max()], [y.min(), y.max()], 'r--', linewidth=2, label='
axes[0].set_xlabel('Actual Values', fontsize=12, fontweight='bold')
axes[0].set_ylabel('Predicted Values', fontsize=12, fontweight='bold')
axes[0].set_title('Random Forest: Actual vs Predicted', fontsize=14, fontweight=
axes[0].legend(fontsize=10)
axes[0].grid(True, alpha=0.3)

# Plot 2: Predictions on original data
# Sort for better visualization
sort_idx = np.argsort(X.flatten())
axes[1].scatter(X, y, color='blue', alpha=0.6, s=100, edgecolors='black', label=
axes[1].plot(X[sort_idx], y_pred_rf[sort_idx], color='green', linewidth=2, marke
             markersize=5, label='RF Predictions')
axes[1].set_xlabel(df.columns[0], fontsize=12, fontweight='bold')
axes[1].set_ylabel(df.columns[1], fontsize=12, fontweight='bold')
axes[1].set_title('Random Forest Regression', fontsize=14, fontweight='bold')
axes[1].legend(fontsize=10)
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```





I) Evaluate Random Forest Model

In [13]:
```
# Calculate metrics
mse_rf = mean_squared_error(y, y_pred_rf)
r2_rf = r2_score(y, y_pred_rf)
mae_rf = mean_absolute_error(y, y_pred_rf)
rmse_rf = np.sqrt(mse_rf)

# Print results
print("="*60)
print("RANDOM FOREST REGRESSION - EVALUATION METRICS")
print("="*60)
print(f"Mean Absolute Error (MAE):       {mae_rf:.2f}")
print(f"Mean Squared Error (MSE):        {mse_rf:.2f}")
print(f"Root Mean Squared Error (RMSE):  {rmse_rf:.2f}")
print(f"R² Score:                        {r2_rf:.4f}")
print("="*60)
```

```
============================================================
RANDOM FOREST REGRESSION - EVALUATION METRICS
============================================================
Mean Absolute Error (MAE):       5755415074.96
Mean Squared Error (MSE):        114499075268875763712.00
Root Mean Squared Error (RMSE):  10700424069.58
R² Score:                        0.9996
============================================================
```

Actual vs Predicted Values Table

In [14]:
```
# Create comparison table
rf_comparison = pd.DataFrame({
    'Actual': y,
    'Predicted': y_pred_rf,
    'Difference': y - y_pred_rf
})

print("\n" + "="*60)
print("RANDOM FOREST - ACTUAL VS PREDICTED VALUES")
print("="*60)
print(rf_comparison.to_string(index=True))
print("="*60)
```

```
============================================================
RANDOM FOREST - ACTUAL VS PREDICTED VALUES
============================================================
            Actual     Predicted    Difference
0      27012159310  2.865896e+10 -1.646802e+09
1      30879471051  3.005119e+10  8.282778e+08
2      29343547254  3.034472e+10 -1.001173e+09
3      34680123880  3.322497e+10  1.455153e+09
4      36018006609  3.568577e+10  3.322344e+08
5      38187786978  3.692639e+10  1.261394e+09
6      32706481878  3.400488e+10 -1.298393e+09
7      32432637099  3.250084e+10 -6.819845e+07
8      32105879383  3.209380e+10  1.207974e+07
9      30876958507  3.132027e+10 -4.433099e+08
10     31622036180  3.150309e+10  1.189435e+08
11     31622036180  3.159995e+10  2.208336e+07
12     94671087100  9.837787e+10 -3.706782e+09
13     99004390549  1.002412e+11 -1.236799e+09
14    107150304082  1.065078e+11  6.425275e+08
15    103192262040  1.055117e+11 -2.319439e+09
16    102954106786  1.098739e+11 -6.919794e+09
17    176520446624  1.500232e+11  2.649729e+10
18    144285884374  1.490670e+11 -4.781091e+09
19    126954702479  1.313011e+11 -4.346418e+09
20    125881395446  1.272976e+11 -1.416242e+09
21    129594318283  1.274887e+11  2.105660e+09
22    120354162559  1.228852e+11 -2.531062e+09
23    120008330730  1.204563e+11 -4.479477e+08
24    118547963195  1.197989e+11 -1.250952e+09
25    121922240736  1.202033e+11  1.718968e+09
26    119548477822  1.196847e+11 -1.362586e+08
27    116945932444  1.176784e+11 -7.325045e+08
28    115721424378  1.169359e+11 -1.214479e+09
29    117038362283  1.161751e+11  8.632814e+08
30    111914219642  1.122847e+11 -3.704945e+08
31    106506412503  1.083131e+11 -1.806710e+09
32    107903218774  1.076539e+11  2.492882e+08
33    106830058199  1.073239e+11 -4.938123e+08
34    107653873240  1.067437e+11  9.101850e+08
35    104530741915  1.038695e+11  6.612054e+08
36     91175889905  9.433804e+10 -3.162148e+09
37     84076027446  8.565005e+10 -1.574025e+09
38     81547103934  8.053107e+10  1.016031e+09
39     56470095631  6.681318e+10 -1.034308e+10
40     64635074417  6.296750e+10  1.667577e+09
41     64592353102  6.124918e+10  3.343171e+09
42     50531274552  5.768804e+10 -7.156765e+09
43     62670533552  5.922506e+10  3.445473e+09
44     58287964169  5.863528e+10 -3.473167e+08
45     54428622271  5.801221e+10 -3.583587e+09
46     58317226695  5.863190e+10 -3.146722e+08
47     57966977414  5.890974e+10 -9.427623e+08
48     60818072872  5.997169e+10  8.463843e+08
49     60597464160  6.047361e+10  1.238556e+08
50     62165489454  6.058585e+10  1.579640e+09
51     60476287474  6.063682e+10 -1.605338e+08
52     64135591641  6.067364e+10  3.461953e+09
53     62451208771  6.067364e+10  1.777570e+09
54     60865237349  6.062423e+10  2.410039e+08
55     60963483229  6.044641e+10  5.170741e+08
```

56	59086909794	5.981977e+10	-7.328578e+08
57	57684748539	5.927919e+10	-1.594444e+09
58	56408494583	5.919590e+10	-2.787406e+09
59	57753264393	5.955575e+10	-1.802485e+09
60	63370149596	6.226615e+10	1.104004e+09
61	65125331605	6.474032e+10	3.850129e+08
62	72539343842	7.180773e+10	7.316122e+08
63	98717264548	8.985647e+10	8.860794e+09
64	101764095132	9.876380e+10	3.000298e+09
65	103499397588	1.002404e+11	3.258985e+09
66	131483603603	1.317717e+11	-2.881190e+08
67	131607066546	1.317754e+11	-1.683599e+08
68	132091080199	1.319503e+11	1.407336e+08
69	131888095908	1.320133e+11	-1.252248e+08
70	132534604575	1.324695e+11	6.513014e+07
71	133112205770	1.331127e+11	-4.826528e+05
72	134046972557	1.339725e+11	7.443664e+07
73	134029099736	1.342126e+11	-1.835286e+08
74	134925773128	1.344965e+11	4.292724e+08
75	134735464409	1.346032e+11	1.322797e+08
76	134476215443	1.345999e+11	-1.236802e+08
77	134614923707	1.345999e+11	1.502803e+07
78	134533228959	1.345999e+11	-6.666672e+07
79	134834521879	1.346070e+11	2.274777e+08
80	134854862271	1.346070e+11	2.478181e+08
81	134516392387	1.345968e+11	-8.045478e+07
82	134294268444	1.345698e+11	-2.754932e+08
83	134996184702	1.338817e+11	1.114524e+09
84	131896484732	1.263671e+11	5.529364e+09
85	82212988326	9.514740e+10	-1.293442e+10
86	83072857411	8.522015e+10	-2.147291e+09
87	84965820546	8.566164e+10	-6.958199e+08
88	102469171865	9.720257e+10	5.266599e+09
89	106814068296	1.009935e+11	5.820544e+09
90	102237956254	1.006276e+11	1.610349e+09
91	100542954271	9.974587e+10	7.970795e+08
92	93689524237	9.504952e+10	-1.360000e+09
93	90835111735	9.410461e+10	-3.269503e+09
94	91664626642	9.410461e+10	-2.439988e+09
95	96521384753	9.420118e+10	2.320205e+09
96	96666279585	9.422816e+10	2.438122e+09
97	92058525423	9.321722e+10	-1.158696e+09
98	90228064474	9.148962e+10	-1.261555e+09
99	89016914128	8.970659e+10	-6.896741e+08
100	88289727172	8.890797e+10	-6.182418e+08
101	85947230592	8.772963e+10	-1.782394e+09
102	86117057807	8.746697e+10	-1.349916e+09
103	86917917758	8.724678e+10	-3.288652e+08
104	84861231154	8.664499e+10	-1.783762e+09
105	80457270390	8.288182e+10	-2.424550e+09
106	70657295066	6.869515e+10	1.962142e+09
107	68783248617	6.842539e+10	3.578579e+08
108	67874168309	6.821294e+10	-3.387746e+08
109	68091847163	6.820893e+10	-1.170867e+08
110	69073746579	6.790813e+10	1.165612e+09
111	65335415630	6.664601e+10	-1.310598e+09
112	65617970340	6.590675e+10	-2.887772e+08
113	60416720550	6.459568e+10	-4.178956e+09
114	65637015222	6.591212e+10	-2.751058e+08
115	73299133597	7.156921e+10	1.729923e+09

116	86805899257	7.960816e+10	7.197738e+09
117	77728230900	7.942284e+10	-1.694608e+09
118	79847451552	7.902457e+10	8.228843e+08
119	80875615979	7.881078e+10	2.064838e+09
120	74280355990	7.520373e+10	-9.233721e+08
121	70634262630	7.351033e+10	-2.876068e+09
122	73609990716	7.374463e+10	-1.346373e+08
123	67795094512	6.642375e+10	1.371342e+09
124	63940452811	6.461595e+10	-6.754939e+08
125	60559882173	6.183291e+10	-1.273027e+09
126	60559544239	6.180362e+10	-1.244071e+09
127	66956018873	6.469613e+10	2.259890e+09
128	62040487716	6.321638e+10	-1.175895e+09
129	59784093139	6.066444e+10	-8.803478e+08
130	60063702645	5.940601e+10	6.576907e+08
131	56570651223	5.681578e+10	-2.451264e+08
132	51627857359	5.215277e+10	-5.249094e+08
133	48758615067	5.039670e+10	-1.638080e+09
134	50895027061	5.071416e+10	1.808626e+08
135	52261508422	5.240645e+10	-1.449457e+08
136	54998650143	5.469539e+10	3.032563e+08
137	61302334914	5.981871e+10	1.483624e+09
138	65208823624	6.433817e+10	8.706511e+08
139	71473878990	6.824462e+10	3.229261e+09
140	64990675207	6.667086e+10	-1.680187e+09
141	68328199216	6.680543e+10	1.522768e+09
142	63558626884	6.511215e+10	-1.553523e+09
143	65702752846	6.519386e+10	5.088914e+08
144	65378810690	6.490175e+10	4.770618e+08
145	58162369962	6.159588e+10	-3.433507e+09
146	53196201967	5.181252e+10	1.383686e+09
147	51441950050	5.102310e+10	4.188479e+08
148	46567238506	4.664035e+10	-7.311539e+07
149	44084667610	4.416468e+10	-8.000848e+07
150	42802406483	4.295477e+10	-1.523626e+08
151	42890781363	4.288891e+10	1.876108e+06
152	43614611203	4.282914e+10	7.854696e+08
153	44640127980	4.255703e+10	2.083099e+09
154	39670379255	4.101966e+10	-1.349282e+09
155	39530294096	4.007775e+10	-5.474516e+08
156	37088103883	4.004517e+10	-2.957062e+09
157	37337921947	4.002231e+10	-2.684386e+09
158	36714100824	3.999722e+10	-3.283117e+09
159	37024539357	3.999722e+10	-2.972679e+09
160	37020476956	4.005524e+10	-3.034766e+09
161	40098011979	4.041789e+10	-3.198766e+08
162	40725445533	4.074162e+10	-1.617626e+07
163	44519399162	4.189812e+10	2.621281e+09
164	44516643247	4.226888e+10	2.247765e+09
165	43926710583	4.229443e+10	1.632277e+09
166	48845464290	4.368833e+10	5.157139e+09
167	53134095174	4.463872e+10	8.495372e+09
168	38523803530	3.890466e+10	-3.808559e+08
169	37940334135	3.889233e+10	-9.519992e+08
170	37852775536	3.876570e+10	-9.129215e+08
171	36599226905	3.793791e+10	-1.338680e+09
172	33988065538	3.319272e+10	7.953463e+08
173	32742732300	3.054550e+10	2.197236e+09
174	30591676631	2.947675e+10	1.114930e+09
175	29070518144	2.916681e+10	-9.628716e+07

176	28750473287	2.913200e+10	-3.815251e+08
177	25381019837	2.913200e+10	-3.750979e+09
178	27851582979	2.913200e+10	-1.280415e+09
179	28629254250	2.913200e+10	-5.027442e+08
180	29850570216	2.914136e+10	7.092084e+08
181	29693577328	2.914136e+10	5.522155e+08
182	29430311678	2.914136e+10	2.889499e+08
183	28423586156	2.914136e+10	-7.177756e+08
184	26791097448	2.914136e+10	-2.350264e+09
185	27499552726	2.914136e+10	-1.641809e+09
186	27635055393	2.914136e+10	-1.506306e+09
187	29374470277	2.915352e+10	2.209483e+08
188	30487497375	2.915352e+10	1.333975e+09
189	28544255442	2.915352e+10	-6.092666e+08
190	28159739672	2.915352e+10	-9.937823e+08
191	29011845137	2.915352e+10	-1.416769e+08
192	29387528608	2.923707e+10	1.504569e+08
193	30434313100	2.972122e+10	7.130963e+08
194	32780289916	3.187340e+10	9.068868e+08
195	34367060400	3.261430e+10	1.752763e+09
196	34656244170	3.263445e+10	2.021796e+09
197	31501631712	3.248635e+10	-9.847229e+08
198	32512825244	3.239138e+10	1.214449e+08
199	31872377591	3.239138e+10	-5.190028e+08
200	31576940909	3.238202e+10	-8.050806e+08
201	38795661031	4.048716e+10	-1.691504e+09
202	39032202917	4.048716e+10	-1.454962e+09
203	39795401292	4.056744e+10	-7.720414e+08
204	42677084758	4.072392e+10	1.953164e+09
205	43920063211	4.072392e+10	3.196143e+09
206	42907941566	4.072392e+10	2.184021e+09
207	41629255403	4.072392e+10	9.053351e+08
208	42627054820	4.072392e+10	1.903135e+09
209	44052671090	4.072392e+10	3.328751e+09
210	43853474600	4.072392e+10	3.129554e+09
211	40813611043	4.068710e+10	1.265158e+08
212	42164630291	4.063961e+10	1.525020e+09
213	40474038206	4.058012e+10	-1.060823e+08
214	39627548688	4.054606e+10	-9.185102e+08
215	39524190406	4.054606e+10	-1.021868e+09
216	39513692928	4.054606e+10	-1.032366e+09
217	40678178342	4.054606e+10	1.321195e+08
218	45022597222	4.054606e+10	4.476538e+09
219	42178326920	4.051010e+10	1.668224e+09
220	39314304729	3.981973e+10	-5.054266e+08
221	39647069757	3.968310e+10	-3.603013e+07
222	38275664506	3.950273e+10	-1.227063e+09
223	37455732920	3.950273e+10	-2.046995e+09
224	35831721141	3.950273e+10	-3.671007e+09
225	35358091194	3.950273e+10	-4.144637e+09
226	36096774231	3.950273e+10	-3.405954e+09
227	35234497538	3.950273e+10	-4.268230e+09
228	34820266362	3.950273e+10	-4.682461e+09
229	36869725756	3.950273e+10	-2.633002e+09
230	39268785117	3.950273e+10	-2.339427e+08
231	40451826533	3.957654e+10	8.752878e+08
232	40521915362	3.957654e+10	9.453766e+08
233	38805644754	3.957654e+10	-7.708940e+08
234	40828338749	3.957654e+10	1.251800e+09
235	40685633133	3.960658e+10	1.079056e+09

236	39002807224	3.960658e+10	-6.037703e+08
237	40175570997	3.960658e+10	5.689935e+08
238	40575391450	3.990835e+10	6.670386e+08
239	43128304118	4.224895e+10	8.793565e+08
240	46163038407	4.284024e+10	3.322796e+09
241	46489462874	4.294539e+10	3.544075e+09
242	44091353530	4.294539e+10	1.145965e+09
243	43711764978	4.294539e+10	7.663769e+08
244	39875504398	4.294539e+10	-3.069884e+09
245	41684479250	4.294539e+10	-1.260909e+09
246	41822348947	4.294539e+10	-1.123039e+09
247	40306059337	4.294539e+10	-2.639329e+09
248	39983260368	4.294539e+10	-2.962128e+09
249	42470868359	4.307984e+10	-6.089694e+08
250	45604217386	4.337430e+10	2.229914e+09
251	43944048450	4.341202e+10	5.320241e+08
252	45164934227	4.341202e+10	1.752910e+09
253	46727007044	4.341202e+10	3.314983e+09
254	47328140796	4.341202e+10	3.916116e+09
255	46175687324	4.341202e+10	2.763663e+09
256	44565289699	4.341202e+10	1.153265e+09
257	44050934569	4.341202e+10	6.389103e+08
258	43324624388	4.341202e+10	-8.739992e+07
259	41926503437	4.314835e+10	-1.221850e+09
260	39245470231	4.123053e+10	-1.985064e+09
261	38871123540	4.043775e+10	-1.566628e+09
262	39368072452	4.019444e+10	-8.263652e+08
263	38511389819	3.983235e+10	-1.320958e+09
264	37130357906	3.827054e+10	-1.140179e+09
265	35839580118	3.666139e+10	-8.218125e+08
266	36050947283	3.597849e+10	7.246117e+07
267	36018729218	3.583662e+10	1.821110e+08
268	34773333455	3.480776e+10	-3.443045e+07
269	34251049159	3.408689e+10	1.641556e+08
270	32885091402	3.252750e+10	3.575941e+08
271	30054566437	3.027634e+10	-2.217776e+08
272	26713039205	2.913921e+10	-2.426174e+09
273	27266536806	2.899074e+10	-1.724205e+09
274	27166235573	2.897536e+10	-1.809121e+09
275	28139019370	2.906108e+10	-9.220620e+08
276	29083031629	2.943484e+10	-3.518078e+08
277	28798239847	2.959802e+10	-7.997765e+08
278	30632565727	3.013452e+10	4.980413e+08
279	33432489203	3.247085e+10	9.616345e+08
280	33732588057	3.376385e+10	-3.125767e+07
281	35139081344	3.417382e+10	9.652644e+08
282	35186230434	3.432010e+10	8.661296e+08
283	36322327786	3.431543e+10	2.006900e+09
284	33019165653	3.359560e+10	-5.764315e+08
285	32839517844	3.337443e+10	-5.349150e+08
286	33046067123	3.340995e+10	-3.638807e+08
287	36127778237	3.446437e+10	1.663412e+09
288	49362415256	4.986056e+10	-4.981436e+08
289	50313759422	5.010583e+10	2.079264e+08
290	50407337295	5.040885e+10	-1.517477e+06
291	54604590697	5.240081e+10	2.203784e+09
292	49119537667	5.044581e+10	-1.326272e+09
293	49843531684	4.982148e+10	2.205546e+07
294	50664071151	4.993963e+10	7.244436e+08
295	49194301547	4.966460e+10	-4.703024e+08

296	50214278815	4.967417e+10	5.401065e+08
297	49541990529	4.866959e+10	8.724038e+08
298	43924500729	4.436155e+10	-4.370481e+08
299	43585236323	4.295781e+10	6.274291e+08
300	41821829819	4.197548e+10	-1.536483e+08
301	39705938167	4.164619e+10	-1.940254e+09
302	44027086762	4.164183e+10	2.385258e+09
303	42039489197	4.147606e+10	5.634302e+08
304	40815856198	4.100497e+10	-1.891128e+08
305	40252061075	4.092104e+10	-6.689825e+08
306	39806000598	4.089781e+10	-1.091808e+09
307	38958543132	4.088894e+10	-1.930399e+09
308	39501995722	4.090964e+10	-1.407640e+09
309	41043955919	4.098683e+10	5.712679e+07
310	42812960478	4.167702e+10	1.135938e+09
311	44169557465	4.335443e+10	8.151235e+08
312	44857348369	4.479948e+10	5.786741e+07
313	45550718234	4.546177e+10	8.895250e+07
314	47600461876	4.638216e+10	1.218303e+09
315	46297426904	4.710369e+10	-8.062620e+08
316	47919066891	4.752006e+10	3.990109e+08
317	47787582906	4.771752e+10	7.006040e+07
318	49223088030	4.876901e+10	4.540784e+08
319	51324642169	5.065518e+10	6.694602e+08
320	53868110319	5.306391e+10	8.042019e+08
321	54988341116	5.490609e+10	8.225236e+07
322	57371727775	5.701393e+10	3.577970e+08
323	61820930580	5.951266e+10	2.308273e+09
324	81188600235	8.513106e+10	-3.942458e+09
325	90848221528	8.860852e+10	2.239700e+09
326	99901840524	9.704591e+10	2.855932e+09
327	96168433816	9.797029e+10	-1.801855e+09
328	105580375065	1.038845e+11	1.695833e+09
329	114764517632	1.126509e+11	2.113572e+09
330	116575417964	1.158507e+11	7.247100e+08
331	115772998970	1.162714e+11	-4.984227e+08
332	118402399303	1.172731e+11	1.129266e+09
333	114593511710	1.157332e+11	-1.139640e+09
334	117196488622	1.168148e+11	3.817026e+08
335	118920109558	1.187283e+11	1.918361e+08
336	119902703361	1.203878e+11	-4.850502e+08
337	131035901773	1.282615e+11	2.774396e+09
338	135760685125	1.343788e+11	1.381844e+09
339	137866371816	1.371025e+11	7.638893e+08
340	140250677149	1.389290e+11	1.321669e+09
341	136262777031	1.374440e+11	-1.181214e+09
342	469081956789	4.463095e+11	2.277244e+10
343	410833729642	4.218453e+11	-1.101153e+10
344	398072362273	4.064370e+11	-8.364645e+09
345	406509667141	4.098991e+11	-3.389438e+09
346	449319672467	4.426475e+11	6.672179e+09
347	470578509837	4.638182e+11	6.760288e+09
348	453657402454	4.587939e+11	-5.136518e+09
349	463961325812	4.637736e+11	1.876893e+08
350	491383415698	4.814669e+11	9.916525e+09
351	460270855696	4.655337e+11	-5.262829e+09
352	422400426810	4.354768e+11	-1.307638e+10
353	440251904598	4.376605e+11	2.591409e+09
354	434589442418	4.336105e+11	9.788986e+08
355	396650266418	4.026004e+11	-5.950171e+09

356	383599270776	3.877042e+11	-4.104968e+09
357	360737208745	3.657870e+11	-5.049837e+09
358	339926526333	3.492062e+11	-9.279705e+09
359	380130751535	3.680091e+11	1.212163e+10
360	361927759996	3.673425e+11	-5.414759e+09
361	377200501098	3.715311e+11	5.669419e+09
362	365007033037	3.687389e+11	-3.731829e+09
363	369892181713	3.694197e+11	4.725109e+08
364	371276388901	3.701327e+11	1.143666e+09
365	378965060085	3.737497e+11	5.215370e+09
366	344402608900	3.522003e+11	-7.797740e+09
367	324603047139	3.317099e+11	-7.106872e+09
368	340633779749	3.331867e+11	7.447060e+09
369	298909133647	3.087736e+11	-9.864451e+09
370	274841392581	2.812858e+11	-6.444381e+09
371	243275173160	2.493347e+11	-6.059523e+09
372	221910710300	2.258143e+11	-3.903617e+09
373	191932089133	2.015368e+11	-9.604667e+09
374	194080140882	1.904428e+11	3.637328e+09
375	170141902664	1.744347e+11	-4.292828e+09
376	144021174996	1.538689e+11	-9.847701e+09
377	161620165475	1.585083e+11	3.111887e+09
378	154636690745	1.620795e+11	-7.442799e+09
379	185929899351	1.790179e+11	6.912017e+09
380	188385368788	1.858276e+11	2.557755e+09
381	174547297195	1.805135e+11	-5.966223e+09
382	181773371643	1.815494e+11	2.240009e+08
383	204674570795	1.927394e+11	1.193517e+10
384	181752571854	1.857385e+11	-3.985902e+09
385	172066715479	1.811666e+11	-9.099843e+09
386	187145392210	1.852415e+11	1.903900e+09
387	183973313426	1.865496e+11	-2.576310e+09
388	202051308905	1.947914e+11	7.259941e+09
389	190925795446	1.918597e+11	-9.339079e+08
390	179381231306	1.828453e+11	-3.464105e+09
391	180807690287	1.815497e+11	-7.420047e+08
392	178351071256	1.827701e+11	-4.419068e+09
393	201846651408	1.963982e+11	5.448443e+09
394	221551825051	2.160805e+11	5.471336e+09
395	226924972846	2.281578e+11	-1.232836e+09
396	255982512392	2.473517e+11	8.630798e+09
397	273176046997	2.659594e+11	7.216618e+09
398	252982938372	2.583536e+11	-5.370679e+09
399	258787210299	2.585536e+11	2.336365e+08
400	274952721830	2.682522e+11	6.700498e+09
401	277106720413	2.731765e+11	3.930190e+09
402	1120790798640	1.133407e+12	-1.261660e+10
403	1141927027995	1.141175e+12	7.519506e+08
404	1180684312843	1.165635e+12	1.504904e+10
405	1139283042322	1.154708e+12	-1.542452e+10
406	1219113870473	1.189112e+12	3.000142e+10
407	1176337572070	1.189933e+12	-1.359576e+10
408	1222487532328	1.213853e+12	8.634255e+09
409	1235342073830	1.232563e+12	2.779356e+09
410	1247129892336	1.247767e+12	-6.367619e+08
411	1292887155870	1.279090e+12	1.379747e+10
412	1255652485402	1.268601e+12	-1.294835e+10
413	1279377240035	1.283039e+12	-3.661772e+09
414	1411784411831	1.370136e+12	4.164879e+10
415	1392133894623	1.382698e+12	9.436370e+09

416	1328187306952	1.334377e+12	-6.190045e+09
417	1292743817185	1.314242e+12	-2.149827e+10
418	1320789656600	1.317971e+12	2.818185e+09
419	1331650042441	1.315438e+12	1.621237e+10
420	1245957258608	1.270725e+12	-2.476810e+10
421	1241424553195	1.263082e+12	-2.165749e+10
422	1367833541906	1.327172e+12	4.066186e+10
423	1359537490700	1.355402e+12	4.135956e+09
424	1339095166990	1.357784e+12	-1.868900e+10
425	1423389166909	1.378822e+12	4.456742e+10
426	1318557335283	1.344493e+12	-2.593556e+10
427	1319112571101	1.333687e+12	-1.457442e+10
428	1370342698734	1.347605e+12	2.273806e+10
429	1328853518501	1.344234e+12	-1.538070e+10
430	1356344086278	1.334724e+12	2.161959e+10
431	1257214674746	1.267829e+12	-1.061476e+10
432	1141967365731	1.180204e+12	-3.823626e+10
433	1174348326306	1.163000e+12	1.134868e+10
434	1167448886976	1.160967e+12	6.482000e+09
435	1140632840146	1.141535e+12	-9.022032e+08
436	1122837101872	1.124772e+12	-1.935029e+09
437	1125755424616	1.118609e+12	7.146863e+09
438	1080359294417	1.099514e+12	-1.915461e+10
439	1052232114383	1.093070e+12	-4.083753e+10
440	1159749665268	1.114341e+12	4.540825e+10
441	1131072923332	1.116640e+12	1.443311e+10
442	1052502317343	1.078315e+12	-2.581292e+10
443	1068703030147	1.073728e+12	-5.024476e+09
444	1060440240689	1.079453e+12	-1.901242e+10
445	1162488430989	1.136379e+12	2.610948e+10
446	1122032006789	1.147109e+12	-2.507712e+10
447	1194262507239	1.170966e+12	2.329622e+10
448	1197482142489	1.174122e+12	2.335995e+10
449	1082003499454	1.132861e+12	-5.085731e+10
450	1141213542641	1.131800e+12	9.413457e+09
451	1154742636820	1.136091e+12	1.865200e+10
452	1096729562516	1.129090e+12	-3.235995e+10
453	1148639496495	1.135753e+12	1.288696e+10
454	1138248921060	1.135908e+12	2.341273e+09
455	1129977053123	1.150507e+12	-2.053001e+10
456	1175303740744	1.162425e+12	1.287832e+10
457	1194574157705	1.187373e+12	7.201373e+09
458	1217524198259	1.212170e+12	5.354149e+09
459	1247772502205	1.241892e+12	5.880016e+09
460	1253923082636	1.253522e+12	4.006442e+08
461	1294650476528	1.280125e+12	1.452514e+10
462	1268793415343	1.270724e+12	-1.930832e+09
463	1253652044544	1.260411e+12	-6.758700e+09
464	968483461189	9.524214e+11	1.606205e+10
465	917391779368	9.309629e+11	-1.357112e+10
466	934181015811	9.362229e+11	-2.041874e+09
467	1035025354275	1.003166e+12	3.185983e+10
468	1054897882872	1.045652e+12	9.246262e+09
469	1037688785718	1.045318e+12	-7.629548e+09
470	1052938231679	1.047458e+12	5.480042e+09
471	1034532073769	1.033513e+12	1.019203e+09
472	985699699453	1.006826e+12	-2.112653e+10
473	1428077647278	1.458510e+12	-3.043214e+10
474	1515056052358	1.499390e+12	1.566641e+10
475	1491912866202	1.501508e+12	-9.595027e+09

476	1555674981945	1.542167e+12	1.350753e+10
477	1617933655326	1.607448e+12	1.048575e+10
478	1612544928908	1.620284e+12	-7.738883e+09
479	1706658678734	1.681220e+12	2.543911e+10
480	1688797640033	1.688592e+12	2.055528e+08
481	1646355681226	1.664992e+12	-1.863627e+10
482	1723825454123	1.659270e+12	6.455570e+10
483	1425666113105	1.505049e+12	-7.938326e+10
484	1362163089821	1.406091e+12	-4.392752e+10
485	1415211249284	1.397361e+12	1.784983e+10
486	1345793124814	1.354916e+12	-9.122967e+09
487	1295640508292	1.305374e+12	-9.733367e+09
488	1293756140156	1.291982e+12	1.773820e+09
489	1259738324967	1.267006e+12	-7.268142e+09
490	1266718331402	1.267278e+12	-5.598929e+08
491	1331617004968	1.305064e+12	2.655299e+10
492	1258880197041	1.280809e+12	-2.192925e+10
493	1274843762484	1.268329e+12	6.514485e+09
494	1243674687778	1.253201e+12	-9.525978e+09
495	1256310379362	1.255777e+12	5.331240e+08
496	1260581840690	1.260668e+12	-8.596691e+07
497	1293977768191	1.283637e+12	1.034060e+10
498	1283043953711	1.284797e+12	-1.752579e+09
499	1286125203529	1.274727e+12	1.139778e+10
500	1166521966780	1.204206e+12	-3.768428e+10
501	1195596156618	1.192940e+12	2.656099e+09
502	1207376984498	1.205009e+12	2.367633e+09
503	1205826065952	1.207278e+12	-1.451858e+09
504	1232096251778	1.217203e+12	1.489282e+10
505	1205040568612	1.204496e+12	5.442977e+08
506	1142546978217	1.159054e+12	-1.650656e+10
507	1158383819925	1.159530e+12	-1.145701e+09
508	1242992677540	1.219033e+12	2.395981e+10
509	1218628189654	1.219484e+12	-8.559382e+08
510	1209699353224	1.202269e+12	7.429960e+09
511	1130526916785	1.154067e+12	-2.353974e+10
512	1185788574236	1.168844e+12	1.694502e+10
513	1215401424790	1.190261e+12	2.514052e+10
514	1098558701287	1.133063e+12	-3.450455e+10
515	1094558501281	1.100227e+12	-5.668762e+09
516	1096137621907	1.087989e+12	8.148673e+09
517	1002738365880	1.034299e+12	-3.156016e+10
518	1002694212657	1.006010e+12	-3.315702e+09
519	999094779637	1.002542e+12	-3.447037e+09
520	1045263707225	1.027783e+12	1.748094e+10
521	1011875390810	1.020881e+12	-9.005292e+09
522	1031699958171	1.026451e+12	5.249305e+09
523	1033575853854	1.029656e+12	3.919816e+09
524	1022932232416	1.025278e+12	-2.345758e+09
525	1068640276819	1.084079e+12	-1.543834e+10
526	1100033958561	1.097578e+12	2.456062e+09
527	1145651324644	1.135474e+12	1.017733e+10
528	1129634860861	1.137960e+12	-8.324936e+09
529	1162869171350	1.155235e+12	7.633782e+09
530	1154600956968	1.158248e+12	-3.647469e+09
531	1191819537403	1.181504e+12	1.031594e+10
532	1168894968872	1.176171e+12	-7.275669e+09
533	1201552629138	1.190957e+12	1.059513e+10
534	978989168213	9.670004e+11	1.198879e+10
535	939813834624	9.495112e+11	-9.697411e+09

536	945121440014	9.457496e+11	-6.281216e+08
537	952939724067	9.521102e+11	8.295431e+08
538	972444259444	9.658004e+11	6.643835e+09
539	946538694166	9.535995e+11	-7.060776e+09
540	920055839679	9.301394e+11	-1.008356e+10
541	915426810766	9.169407e+11	-1.513856e+09
542	911942226074	9.093537e+11	2.588534e+09
543	882110227742	8.935756e+11	-1.146538e+10
544	883889283026	8.807010e+11	3.188311e+09
545	860648467813	8.648578e+11	-4.209366e+09
546	850624773935	8.575038e+11	-6.879015e+09
547	873468397121	8.840995e+11	-1.063107e+10
548	926680428823	9.043200e+11	2.236039e+10
549	737208255653	7.351416e+11	2.066656e+09
550	730965512818	7.325821e+11	-1.616562e+09
551	743160005479	7.379527e+11	5.207290e+09
552	711307749066	7.238535e+11	-1.254573e+10
553	739787480138	7.357595e+11	4.027998e+09
554	772021077454	7.598331e+11	1.218798e+10
555	769112095175	7.636233e+11	5.488845e+09
556	472352089335	5.053196e+11	-3.296748e+10
557	535791881214	5.244719e+11	1.132002e+10
558	534358535939	5.352364e+11	-8.778864e+08
559	564504877209	5.527624e+11	1.174252e+10
560	542447601238	5.469389e+11	-4.491273e+09
561	545553224952	5.435956e+11	1.957638e+09
562	530566192050	5.381579e+11	-7.591726e+09
563	546748231695	5.386535e+11	8.094693e+09
564	512247998653	5.221803e+11	-9.932345e+09
565	521787755758	5.230819e+11	-1.294188e+09
566	557569968008	5.454427e+11	1.212730e+10
567	554955460490	5.531593e+11	1.796189e+09
568	535028397294	5.443552e+11	-9.326849e+09
569	575424665370	5.658663e+11	9.558333e+09
570	573176523057	5.706815e+11	2.495019e+09
571	561829189603	5.644348e+11	-2.605590e+09
572	549984522473	5.558997e+11	-5.915214e+09
573	576317141994	5.697074e+11	6.609763e+09
574	589338979811	5.883744e+11	9.646163e+08
575	608244259425	6.054066e+11	2.837631e+09
576	633498175881	6.260828e+11	7.415369e+09
577	653201282467	6.413542e+11	1.184708e+10
578	1336820932083	1.356633e+12	-1.981222e+10
579	1391620119595	1.377457e+12	1.416328e+10
580	1392647726905	1.386631e+12	6.016436e+09
581	1379132390635	1.373804e+12	5.328336e+09
582	1313809974602	1.326458e+12	-1.264774e+10
583	1295367179028	1.295049e+12	3.184232e+08
584	1169605447748	1.210735e+12	-4.112984e+10
585	1169973963483	1.168700e+12	1.273565e+09
586	1073807723879	1.105773e+12	-3.196507e+10
587	1133897640660	1.116266e+12	1.763204e+10
588	1131853006387	1.118751e+12	1.310212e+10
589	1062274334806	1.070107e+12	-7.833016e+09
590	962228666957	9.982225e+11	-3.599387e+10
591	1006309341765	9.971040e+11	9.205378e+09
592	969075197084	9.829800e+11	-1.390478e+10
593	1035058800935	1.020369e+12	1.468993e+10
594	1060943859304	1.055309e+12	5.634820e+09
595	1068839906210	1.071282e+12	-2.442021e+09

```
596  1142212011366  1.124662e+12  1.754990e+10
597  1173640065343  1.161474e+12  1.216623e+10
598  1193186184333  1.185278e+12  7.907754e+09
599  1168588460527  1.176111e+12 -7.522505e+09
600  1197344227961  1.186841e+12  1.050346e+10
601  1231050193601  1.232945e+12 -1.894528e+09
602  1248957308918  1.242077e+12  6.879958e+09
603  1235525649029  1.238746e+12 -3.220270e+09
604  1258707403682  1.248712e+12  9.995352e+09
============================================================
```

Final Comparison & Conclusion

Complete Model Comparison Table

In [15]:
```python
# Compile all results
all_results = []

# Linear Regression
all_results.append({
    'Model': 'Linear Regression',
    'MAE': mae_linear,
    'MSE': mse_linear,
    'RMSE': rmse_linear,
    'R2_Score': r2_linear
})

# Polynomial Regression (all degrees)
all_results.extend(poly_results)

# Random Forest
all_results.append({
    'Model': 'Random Forest',
    'MAE': mae_rf,
    'MSE': mse_rf,
    'RMSE': rmse_rf,
    'R2_Score': r2_rf
})

# Create final comparison DataFrame
final_comparison_df = pd.DataFrame(all_results)

# Sort by R2 Score (descending)
final_comparison_df = final_comparison_df.sort_values('R2_Score', ascending=Fals

print("\n" + "="*80)
print("COMPLETE MODEL COMPARISON - ALL MODELS")
print("="*80)
print(final_comparison_df.to_string(index=False))
print("="*80)
```

```
================================================================================
COMPLETE MODEL COMPARISON - ALL MODELS
================================================================================
                Model          MAE          MSE         RMSE  R2_Score
        Random Forest 5.755415e+09 1.144991e+20 1.070042e+10  0.999550
Polynomial (Degree 7) 1.566681e+11 4.345542e+22 2.084596e+11  0.829392
Polynomial (Degree 4) 1.510331e+11 5.221696e+22 2.285103e+11  0.794994
Polynomial (Degree 6) 1.791979e+11 5.380580e+22 2.319608e+11  0.788756
Polynomial (Degree 3) 1.738529e+11 5.460133e+22 2.336693e+11  0.785633
Polynomial (Degree 5) 1.747861e+11 5.720504e+22 2.391757e+11  0.775410
Polynomial (Degree 2) 2.321532e+11 8.452630e+22 2.907341e+11  0.668146
    Linear Regression 2.448638e+11 8.614642e+22 2.935071e+11  0.661785
================================================================================
```

Identify Best Model

In [16]:

```python
# Find best model based on lowest RMSE
best_model_rmse = final_comparison_df.loc[final_comparison_df['RMSE'].idxmin()]

# Find best model based on highest R2 Score
best_model_r2 = final_comparison_df.loc[final_comparison_df['R2_Score'].idxmax()]

print("\n" + "="*80)
print("BEST MODEL IDENTIFICATION")
print("="*80)
print("\nBased on Lowest RMSE:")
print(f"  Model: {best_model_rmse['Model']}")
print(f"  RMSE:  {best_model_rmse['RMSE']:.2f}")
print(f"  R²:    {best_model_rmse['R2_Score']:.4f}")

print("\nBased on Highest R² Score:")
print(f"  Model: {best_model_r2['Model']}")
print(f"  R²:    {best_model_r2['R2_Score']:.4f}")
print(f"  RMSE:  {best_model_r2['RMSE']:.2f}")
print("="*80)
```

```
================================================================================
BEST MODEL IDENTIFICATION
================================================================================

Based on Lowest RMSE:
  Model: Random Forest
  RMSE:  10700424069.58
  R²:    0.9996

Based on Highest R² Score:
  Model: Random Forest
  R²:    0.9996
  RMSE:  10700424069.58
================================================================================
```

Visual Comparison of Model Performance

In [17]:

```python
# Create visual comparison
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Plot 1: RMSE Comparison
models = final_comparison_df['Model']
```

```
rmse_values = final_comparison_df['RMSE']
colors = ['red' if x == rmse_values.min() else 'skyblue' for x in rmse_values]

axes[0].barh(models, rmse_values, color=colors, edgecolor='black')
axes[0].set_xlabel('RMSE (Lower is Better)', fontsize=12, fontweight='bold')
axes[0].set_title('Model Comparison: RMSE', fontsize=14, fontweight='bold')
axes[0].grid(axis='x', alpha=0.3)

# Plot 2: R² Score Comparison
r2_values = final_comparison_df['R2_Score']
colors = ['green' if x == r2_values.max() else 'lightcoral' for x in r2_values]

axes[1].barh(models, r2_values, color=colors, edgecolor='black')
axes[1].set_xlabel('R² Score (Higher is Better)', fontsize=12, fontweight='bold'
axes[1].set_title('Model Comparison: R² Score', fontsize=14, fontweight='bold')
axes[1].grid(axis='x', alpha=0.3)

plt.tight_layout()
plt.show()
```

📊 CONCLUSION

Best Model Selection

Based on comprehensive evaluation using multiple metrics (MAE, MSE, RMSE, and R^2 Score), the analysis reveals:

Key Findings:

1. **Model Performance Trend:**
   - Simple Linear Regression provides a baseline performance
   - Polynomial Regression shows improved performance as degree increases (up to a point)
   - Higher-degree polynomials may overfit, showing excellent training performance but potential generalization issues
   - Random Forest Regression typically provides robust predictions with good generalization
2. **Best Model Criteria:**

- **Lowest RMSE:** Indicates the model with smallest prediction errors
- **Highest R^2 Score:** Indicates the model that best explains variance in the data

3. **Justification for Best Model:**

   - The model with the **highest R^2 score** and **lowest RMSE** is considered the best
   - For simple datasets, Linear Regression often performs well due to its simplicity and interpretability
   - For complex non-linear relationships, Polynomial Regression (optimal degree) or Random Forest may excel
   - Random Forest is robust against overfitting and handles non-linearity well

4. **Trade-offs:**

   - **Linear Regression:** Simple, interpretable, fast, but limited to linear relationships
   - **Polynomial Regression:** Can model non-linear relationships, but risk of overfitting with high degrees
   - **Random Forest:** Excellent for complex patterns, robust, but less interpretable

Final Recommendation:

The **best model** is the one that achieves the optimal balance between:

- High R^2 Score (close to 1.0)
- Low RMSE
- Model complexity vs. interpretability
- Generalization capability (avoiding overfitting)

Check the comparison table above to see which specific model performed best on this dataset!
